

ОНТОЛОГИЯ РЕАЛЬНОСТИ

Итоговая рабочая версия 3.1

Полная человекочитаемая спецификация

Entity, 10 основных классов, вычисляемый Subject, свойства, методы, ограничения, связи, provenance, эволюция модели, матрица потребностей и живая онтология

Версия 3.1 включает без сокращений весь текст рабочей версии 2.0. Новые разделы 9–12 фиксируют последующие согласованные расширения и явно заменяют только указанные прежние рабочие формулировки.

ОНТОЛОГИЯ РЕАЛЬНОСТИ

Entity, десять основных классов, методы, ограничения и эволюция модели

Рабочая версия 2.0

1. Архитектура онтологии

Базовой единицей является Entity. Один Entity может одновременно принадлежать нескольким классам; для нечётких классов принадлежность задаётся $\mu \in [0,1]$. Subject не является отдельным фундаментальным классом: субъектность вычисляется из свойств Entity. Полное описание класса включает свойства, методы, ограничения и связи.

$$Class = Properties + Methods + Constraints + Relations$$

Каждое вычисленное или внешнее значение хранится вместе с метаданными происхождения:

$$p_j = (value, unit, time, source, method, version, confidence)$$

2. Базовый класс Entity

Entity — любой различимый объект реальности, который существует или существовал как идентифицируемая единица и может быть представлен в онтологии.

2.1. Свойства Entity

Свойство	Содержание	Тип	Статус
ID	Уникальный идентификатор Entity	string	обязательно
Name / Label	Человекочитаемое имя или метка	string	необязательно
ClassMembership	Множество классов, к которым относится Entity	set[class]	обязательно
μ_k	Степень fuzzy-принадлежности к нечёткому классу	number [0,1]	если применимо
Boundary	Правило, определяющее границу объекта относительно среды	rule / geometry	концептуально обязательно
Persistence	Критерий сохранения идентичности во времени	rule	концептуально обязательно
Components	Составные Entity	list[EntityID]	если применимо
Properties	Набор специализированных свойств	map	обязательно
State X(t)	Текущее состояние объекта	State reference	если измеримо
Location L(t)	Физическая или логическая локализация	Location reference	если применимо
t_birth / t_end	Время возникновения и прекращения существования	datetime	желательно

Status	active / terminated / unknown / other	enum	обязательно
Origin	Происхождение или механизм возникновения	reference / text	желательно
Provenance	Происхождение данных и изменений	Provenance record	обязательно для данных
Confidence c	Уверенность в идентификации или оценке свойства	number [0,1]	для оценочных данных

2.2. Методы Entity

Метод	Назначение
create()	Создать новое представление Entity при наличии достаточных данных об идентичности.
updateProperty()	Обновить значение свойства с сохранением времени, источника и предыдущего значения.
updateState()	Связать Entity с новым State и сохранить историю состояний.
classify() / reclassify()	Добавить, удалить или изменить степень принадлежности к классу.
link() / unlink()	Создать или прекратить типизированную связь с другим Entity.
merge()	Объединить записи, если доказано, что они описывают один и тот же объект.
split()	Разделить Entity, если ранее единый объект оказался несколькими различными объектами.
terminate()	Зафиксировать прекращение существования Entity.
updateConfidence()	Пересчитать достоверность данных при появлении новых источников.
traceProvenance()	Получить происхождение текущего состояния, свойства или классификации.

2.3. Ограничения Entity

Ограничение	Правило
Unique ID	ID каждого Entity уникален в пределах версии онтологии.
Valid fuzzy range	Для fuzzy-принадлежности и confidence: $0 \leq \text{value} \leq 1$.
Temporal consistency	Если t_{end} задан, то $t_{\text{birth}} \leq t_{\text{end}}$; состояние после t_{end} не создаётся без явного правила наследования/архивирования.
Boundary defined	Entity должен иметь определимую концептуальную границу, даже если она нечёткая или динамическая.
Persistence rule	Для продолжительно существующих объектов должен быть задан критерий тождества во времени.
Existing references	Components, State, Location и иные ссылки должны указывать на существующие или явно внешние Entity.
No silent overwrite	Обновление свойства не уничтожает предыдущее значение; создаётся история изменений.
Provenance required	Любое внешнее или вычисленное значение должно иметь source/method/version/time.

Confidence semantics	Низкая confidence не превращается автоматически в низкое value: значение и достоверность хранятся отдельно.
Class consistency	Специализированные классы не могут отменять базовые ограничения Entity без явного правила исключения.

3. Десять специализированных классов

Каждый класс наследует все базовые свойства, методы и ограничения Entity и добавляет специализированные элементы. Методы здесь понимаются как формально допустимые операции/преобразования, а не обязательно как программные функции.

3.1. Resource

То, что может быть использовано для удовлетворения потребностей или как вход в процесс.

Дополнительные свойства

Свойство	Содержание
Quantity	Количество/объём ресурса
Unit	Единица измерения
Availability	Фактическая доступность
Accessibility	Физическая/правовая доступность
Scarcity	Степень дефицитности
Renewability	Возобновляемость
RegenerationRate	Скорость восполнения
DepletionRate	Скорость истощения
Substitutability	Возможность замещения
Quality	Пригодность/качество
ExtractionCost	Стоимость получения
TransformationCost	Стоимость преобразования
StorageCost	Стоимость хранения
Transportability	Перемещаемость
Storability	Возможность хранения
Owner/Controller	Владелец или контролирующий Entity
UseRestrictions	Ограничения использования
Externalities	Внешние эффекты использования

Методы

Метод	Назначение
measureAvailability()	Оценить доступный объём в заданных времени и локации.
allocate()	Закрепить часть ресурса за Entity/Process/Need.
reserve()	Временно вывести объём из свободного распределения.
transfer()	Изменить владельца/контролёра или место ресурса.
consume()	Уменьшить доступное количество вследствие использования.

transform()	Преобразовать ресурс в другой Resource/Product через Process.
replenish()	Увеличить количество вследствие восполнения.
estimateScarcity()	Рассчитать scarcity относительно доступности и потребности.
matchNeed()	Определить пригодность ресурса для удовлетворения Need.

Ограничения

Ограничение	Правило
Non-negative quantity	Quantity ≥ 0 , кроме явно определённых бухгалтерских представлений, которые не являются физическим запасом.
Unit required	Количественный ресурс должен иметь единицу измерения.
Conservation rules	Для материальных/энергетических ресурсов преобразования должны соблюдать применимые законы сохранения.
Allocation limit	Сумма одновременных аллокаций не превышает доступное количество без явного механизма кредита/будущего ресурса.
Access $\neq$ existence	Наличие ресурса не означает его доступность; Availability и Accessibility различаются.
Renewability bounded	Renewability и связанные fuzzy-параметры лежат в [0,1].
Traceable transformation	Transform должен ссылаться на Process и формировать происхождение результата.
Temporal stock consistency	Изменение запаса согласуется с inflow, outflow, transformation и measurement adjustments.

Ключевые связи

- locatedAt -> Location
- ownedOrControlledBy -> Entity
- usedBy -> Process
- satisfies -> Need
- transformedInto -> Resource/Product
- regulatedBy -> LegalConstruct

3.2. Product

Результат процесса деятельности, обладающий функцией и потенциальной полезностью.

Дополнительные свойства

Свойство	Содержание
Producer	Entity-производитель
OutputOfProcess	Процесс происхождения
Composition	Состав/компоненты
Function	Назначение
TargetNeed	Потребности, которые способен удовлетворять
Quantity	Количество
Unit	Единица измерения
Quality	Качество

Performance	Функциональная результативность
Lifetime	Срок службы
Perishability	Скорость потери полезности
Reusability	Повторное использование
Repairability	Ремонтопригодность
Recyclability	Перерабатываемость
Substitutability	Заменяемость
ProductionCost	Стоимость производства
UseCost	Стоимость эксплуатации
Availability	Доступность
Ownership	Право собственности/контроля
Externalities	Побочные эффекты

Методы

Метод	Назначение
produce()	Создать экземпляр Product как выход Process.
evaluateQuality()	Оценить качество по определённым метрикам.
use()	Использовать продукт для функции/удовлетворения Need.
degrade()	Обновить качество/срок полезности во времени.
repair()	Восстановить часть функциональности через Process.
reuse()	Повторно применить продукт без полного преобразования.
recycle()	Преобразовать продукт в Resource/Product.
retire()	Вывести продукт из использования.
matchNeed()	Оценить соответствие конкретной Need.

Ограничения

Ограничение	Правило
Origin required	Product должен иметь Producer/Process или иное объяснимое происхождение.
Functional definition	Для активного Product должна быть задана хотя бы одна функция или целевое применение.
Composition references	Компоненты, если заданы как Entity, должны существовать и иметь отношения partOf.
Quality provenance	Оценка Quality должна иметь метод и provenance.
Lifetime consistency	Возраст продукта не может превышать момент вывода/разрушения без статуса historical/archived.
No double counting	При превращении Product в Resource система должна предотвращать двойной учёт одного физического объекта.
Need matching explicit	Связь с Need требует критерия пригодности, а не только текстового совпадения.

Ключевые связи

- producedBy -> Process

- producedByEntity -> Entity
- contains -> Entity
- satisfies -> Need
- usedIn -> Process
- locatedAt -> Location
- ownedBy -> Entity

3.3. Infrastructure

Долговременно существующая система, обеспечивающая возможность или пропускную способность других процессов.

Дополнительные свойства

Свойство	Содержание
Function	Инфраструктурная функция
Capacity	Максимальная мощность
Throughput	Пропускная способность
Utilization	Текущая загрузка
Coverage	Область охвата
Accessibility	Доступность
Reliability	Надёжность
Redundancy	Резервирование
Wear	Износ
Lifetime	Ожидаемый срок службы
MaintenanceNeed	Потребность в обслуживании
MaintenanceCost	Стоимость обслуживания
Upgradeability	Возможность модернизации
Dependencies	Зависимости
Bottlenecks	Ограничивающие узлы
Owner/Operator	Владелец/оператор
OperatingCost	Эксплуатационные расходы
EnergyUse	Потребление энергии
Resilience	Устойчивость к сбоям
Externalities	Внешние эффекты

Методы

Метод	Назначение
provideCapacity()	Предоставить инфраструктурную мощность Process/Entity.
measureUtilization()	Рассчитать текущую загрузку.
maintain()	Провести обслуживание, изменив Wear/Reliability.
repair()	Восстановить работоспособность после отказа.
upgrade()	Изменить Capacity/Throughput/Function.
decommission()	Вывести инфраструктуру из эксплуатации.

detectBottleneck()	Определить ограничивающий элемент.
estimateResilience()	Оценить устойчивость к заданному классу нарушений.

Ограничения

Ограничение	Правило
Capacity bound	Utilization не должна превышать техническую Capacity без статуса overload.
Dependency consistency	Критические зависимости должны ссылаться на существующие Entity/Infrastructure/Resource.
Maintenance trace	Изменения Wear после обслуживания должны иметь Event/Process provenance.
Coverage semantics	Coverage задаётся географически, логически или по населению и не смешивается без явного преобразования.
Operational status	Capacity/Throughput активной эксплуатации не используются после decommission time.
Reliability scale	Fuzzy Reliability/Resilience хранятся в [0,1] с методом оценки.

Ключевые связи

- supports -> Process
- serves -> Entity/Location
- dependsOn -> Infrastructure/Resource
- operatedBy -> Entity
- locatedAt -> Location
- satisfies -> Need

3.4. Location

Пространственный Entity, задающий место, границу и контекст существования других Entity.

Дополнительные свойства

Свойство	Содержание
Coordinates	Координаты/геометрия
Boundary	Пространственная граница
Area/Volume	Площадь/объём
ParentLocation	Включающая территория
ContainedEntities	Объекты внутри
Accessibility	Доступность
Connectivity	Связность
Population	Население
Density	Плотность
EnvironmentalState	Экологические параметры
Climate	Климатические параметры
NaturalResources	Природные ресурсы
Infrastructure	Связанная инфраструктура
LegalRegime	Юрисдикция/режим

RiskProfile	Риски
LandUse	Использование
Capacity	Вместимость/несущая способность
TemporalAccessibility	Доступность во времени

Методы

Метод	Назначение
contains()	Проверить пространственное включение Entity.
locate()	Определить положение Entity относительно Location.
measureDistance()	Рассчитать расстояние по определённой метрике.
measureAccessibility()	Оценить достижимость с учётом сети, времени и ограничений.
aggregateState()	Агрегировать состояния Entity внутри территории.
updateBoundary()	Изменить геометрию/границы с versioning.
estimateCapacity()	Оценить допустимую нагрузку/вместимость.

Ограничения

Ограничение	Правило
Geometry validity	Boundary должна быть геометрически валидной для выбранной модели пространства.
Hierarchy consistency	ParentLocation не может создавать циклическую пространственную иерархию.
Temporal boundary	Изменение границы требует времени действия версии.
Coordinate reference	Координаты должны указывать систему отсчёта/CRS или иной логический координатный контекст.
Contained entity time	Пространственное включение относится к конкретному времени/интервалу.
Population provenance	Агрегированные показатели населения должны иметь дату и источник.

Ключевые связи

- contains -> Entity
- partOf -> Location
- connectedTo -> Location
- governedBy -> LegalConstruct
- hasInfrastructure -> Infrastructure
- containsResource -> Resource

3.5. Information

Содержательно интерпретируемый объект данных/знания, который может храниться, передаваться и использоваться.

Дополнительные свойства

Свойство	Содержание
Content	Содержание
Format	Формат

Source	Первичный источник
Author/Generator	Создатель
CreationTime	Время создания
Validity	Соответствие действительности
Reliability	Надёжность
Confidence	Уверенность оценки
Freshness	Актуальность
Version	Версия
Granularity	Детализация
Completeness	Полнота
Consistency	Непротиворечивость
Reproducibility	Воспроизводимость
AccessRights	Права доступа
Ownership	Владение/контроль
Sensitivity	Конфиденциальность
Dependencies	Зависимости
SemanticLinks	Семантические связи
DecayRate	Скорость устаревания

Методы

Метод	Назначение
createVersion()	Создать новую версию без уничтожения предыдущей.
validate()	Проверить структуру, типы и логические правила.
verifySource()	Проверить доступную цепочку происхождения.
updateConfidence()	Пересчитать confidence при новых доказательствах.
derive()	Создать производный информационный объект с указанием метода.
linkSemantically()	Добавить типизированную семантическую связь.
restrictAccess()	Изменить правила доступа.
archive()	Перевести устаревшую версию в архивный статус.

Ограничения

Ограничение	Правило
Version immutability	Опубликованная версия не перезаписывается; исправление создаёт новую версию.
Source traceability	Производная информация должна ссылаться на исходные данные и метод.
Access compliance	Доступ и распространение должны соответствовать LegalConstruct.
Confidence separated	Reliability источника, Validity содержания и Confidence оценки не отождествляются.
No provenance loss	Трансформация не должна удалять происхождение входных

	данных.
Timestamp required	Для меняющейся информации должен быть указан момент актуальности.

Ключевые связи

- derivedFrom -> Information
- generatedBy -> Entity/Process
- describes -> Entity/State/Event
- usedBy -> Process
- regulatedBy -> LegalConstruct

3.6. LegalConstruct

Созданная институциональными правилами абстрактная конструкция прав, обязательств или ограничений.

Дополнительные свойства

Свойство	Содержание
Type	Тип конструкции
Jurisdiction	Юрисдикция
RightsHolder	Носитель права
ObligatedParty	Обязанная сторона
ObjectOfRight	Объект права/обязательства
Rights	Права
Obligations	Обязательства
Restrictions	Ограничения
ValidityStart	Начало действия
ValidityEnd	Окончание действия
ActivationConditions	Условия вступления в силу
TerminationConditions	Условия прекращения
Transferability	Передаваемость
Divisibility	Делимость
Enforceability	Исполнимость
Priority	Приоритет
Collateral	Обеспечение
Value	Экономическая оценка
Issuer/Authority	Эмитент/орган
VerificationSource	Источник подтверждения

Методы

Метод	Назначение
activate()	Перевести в действующее состояние при выполнении условий.
terminate()	Прекратить действие согласно условиям.

transfer()	Изменить RightsHolder/ObligatedParty при допустимой передаваемости.
enforce()	Запустить предусмотренный механизм исполнения.
verifyValidity()	Проверить действительность во времени и юрисдикции.
amend()	Создать новую версию условий с историей изменений.
attachCollateral()	Связать обеспечение.
resolvePriority()	Определить приоритет конкурирующих прав по заданному правилу.

Ограничения

Ограничение	Правило
Jurisdiction required	Для правовых эффектов должна быть определена юрисдикция или система правил.
Temporal validity	Действие ограничено ValidityStart/ValidityEnd и условиями.
Existing parties	RightsHolder, ObligatedParty и ObjectOfRight должны быть идентифицируемыми Entity.
Transfer only if allowed	Transfer возможен только если Transferability допускает его.
No silent amendment	Изменение условий создаёт новую версию/историю.
Authority provenance	Действительность должна быть связана с источником полномочий/правил.
Conflict resolution	Конфликтующие конструкции требуют явного правила Priority или неопределённого статуса.

Ключевые связи

- grantsRightTo -> Entity
- imposesObligationOn -> Entity
- appliesTo -> Entity/Process/Resource
- issuedBy -> Entity
- securedBy -> Entity/Resource
- governedIn -> Location

3.7. State

Формальное описание значений релевантных переменных Entity в момент или интервал времени.

Дополнительные свойства

Свойство	Содержание
SubjectEntity	Entity, состояние которого описывается
Variables	Переменные состояния
Values	Значения
Units	Единицы
Timestamp	Момент/интервал
ReferenceState	Референтное состояние
DesiredState	Желаемое состояние
Deviation	Отклонение
Stability	Устойчивость

Trend	Направление изменения
RateOfChange	Скорость изменения
Thresholds	Критические пороги
Uncertainty	Неопределённость
MeasurementMethod	Метод измерения
Source	Источник
Confidence	Достоверность
Dependencies	Определяющие факторы

Методы

Метод	Назначение
observe()	Создать State на основании наблюдений/измерений.
compare()	Сопоставить два состояния одного совместимого объекта/параметра.
computeDeviation()	Рассчитать отклонение от Reference/Desired.
estimateTrend()	Определить тренд по последовательности State.
detectThresholdCrossing()	Зафиксировать переход заданного порога.
interpolate()	Оценить состояние между наблюдениями с явной неопределённостью.
aggregate()	Сформировать агрегированное состояние группы/Location.

Ограничения

Ограничение	Правило
Subject required	State всегда относится к одному или явно определённому агрегату Entity.
Timestamp required	State без временной привязки недопустим.
Dimension consistency	Сравниваемые значения должны быть размерностно совместимы или иметь преобразование.
No future fact	Прогноз не маркируется как наблюдаемое состояние; требуется отдельный status/method.
Confidence required for inferred	Интерполированные/вычисленные значения обязаны иметь confidence и method.
Immutable observation	Зафиксированное наблюдение не перезаписывается; корректировка создаёт новую запись.

Ключевые связи

- stateOf -> Entity
- measuredBy -> Entity/Process
- derivedFrom -> Information
- changedBy -> Event/Process
- comparedTo -> State

3.8. Event

Локализованный во времени факт, приводящий или соответствующий изменению состояния одного или нескольких Entity.

Дополнительные свойства

Свойство	Содержание
EventType	Тип события
StartTime	Начало
EndTime	Окончание
Location	Локализация
Participants	Участники
Trigger	Триггер
Causes	Причины
Preconditions	Предусловия
StateBefore	Состояние до
StateChange ΔX	Изменение
StateAfter	Состояние после
Consequences	Последствия
Magnitude	Масштаб
Probability	Вероятность для прогнозируемого события
Reversibility	Обратимость
AffectedEntities	Затронутые Entity
Externalities	Побочные эффекты
Evidence	Подтверждающие данные
Confidence	Уверенность

Методы

Метод	Назначение
record()	Зафиксировать произошедшее событие и evidence.
predict()	Создать прогнозируемый Event с probability и методом.
linkCause()	Связать возможную/подтверждённую причину с типом уверенности.
applyStateChange()	Сформировать StateAfter на основании подтверждённого изменения.
assessMagnitude()	Оценить масштаб по заданной шкале.
reverse()	Зафиксировать обратное действие, если физически/логически возможно.
aggregateEvents()	Объединить серию событий в более крупное событие без потери исходных ссылок.

Ограничения

Ограничение	Правило
Temporal order	$StartTime \leq EndTime$, если EndTime задан.
Observed vs predicted	Произошедшее и прогнозируемое событие имеют разные статусы.
Evidence provenance	Observed Event должен иметь evidence/source или явно низкую confidence.

Cause $\neq$ correlation	Связь cause должна отличаться от temporalAssociation/correlation.
State consistency	StateAfter должен согласовываться с StateBefore + StateChange в пределах модели.
Participants exist	Участники и AffectedEntities должны быть Entity.
No duplicate event	Дедупликация должна учитывать время, место, участников и provenance.

Ключевые связи

- occursAt -> Location
- involves -> Entity
- changes -> State
- causedBy -> Event/Process/Entity
- evidencedBy -> Information

3.9. Process

Протяжённое во времени преобразование входов в выходы и/или изменение состояний.

Дополнительные свойства

Свойство	Содержание
ProcessType	Тип процесса
Inputs	Входы
Outputs	Выходы
Executor	Исполнитель
Participants	Участники
StartTime	Начало
EndTime	Окончание
Duration	Продолжительность
Location	Локализация
ResourceUse	Используемые ресурсы
EnergyUse	Энергозатраты
InformationUse	Информация
Rules/Algorithm	Правила/алгоритм
Capacity	Максимальная мощность
Throughput	Фактическая пропускная способность
Efficiency	Эффективность
Quality	Качество результата
Cost	Полные затраты
Bottlenecks	Ограничители
Dependencies	Зависимости
SideEffects	Побочные эффекты
Externalities	Внешние эффекты

Reversibility	Обратимость
ControlParameters	Управляющие параметры
OutcomeStateChange	Изменение состояний

Методы

Метод	Назначение
start()	Перевести процесс в active при выполнении предусловий.
pause() / resume()	Приостановить/возобновить, если тип процесса это допускает.
stop()	Завершить процесс.
consumeInputs()	Учесть использование входов/ресурсов.
produceOutputs()	Создать выходные Product/Resource/Information/State.
measureEfficiency()	Рассчитать эффективность по определённой функции.
detectBottleneck()	Определить ограничивающий фактор.
adjustControl()	Изменить управляющий параметр с записью причины.
simulate()	Рассчитать прогноз результата без изменения фактического состояния.
audit()	Восстановить цепочку inputs → transformations → outputs.

Ограничения

Ограничение	Правило
Input existence	Все обязательные Inputs должны существовать или быть явно виртуальными/прогнозируемыми.
Temporal consistency	$StartTime \leq EndTime$; Duration согласуется с интервалом.
Resource accounting	Потребление ресурсов должно отражаться в их State/Quantity.
Output provenance	Каждый Output имеет ссылку на Process и использованные Inputs.
Simulation separation	Результат simulate не изменяет фактические State без отдельного commit/action.
Capacity bound	$Throughput > Capacity$ требует overload state или пересмотра Capacity.
Algorithm version	Если результат зависит от Rules/Algorithm, должна быть зафиксирована версия.
No hidden side effects	Известные SideEffects/Externalities сохраняются как часть результата, а не игнорируются.

Ключевые связи

- uses -> Resource/Information
- produces -> Product/Resource/Information
- executedBy -> Entity
- occursAt -> Location
- changes -> State
- supportedBy -> Infrastructure
- regulatedBy -> LegalConstruct
- satisfies -> Need

3.10. Need

Вычисляемая структура, описывающая значимый разрыв между фактическим и желаемым/референтным состоянием носителя или связанного с ним Entity.

Дополнительные свойства

Свойство	Содержание
Holder	Носитель потребности
TargetEntity/Object	Объект потребности
ActualState	Фактическое X_actual
DesiredState	Желаемое X_desired
ReferenceState	Нормативный уровень
Deficit D	Величина разрыва
Urgency U	Срочность
Significance V	Значимость
Satisfiability F	Удовлетворяемость
SaturationLevel	Уровень насыщения
MinimumThreshold	Минимально допустимый уровень
PriorityConstraints	Ограничения приоритета
TemporalHorizon	Временной горизонт
Duration	Продолжительность
Recurrence	Повторяемость
AffectedPopulation	Масштаб населения/субъектов
SpatialDistribution	Пространственное распределение
Substitutability	Альтернативные способы удовлетворения
Complementarity	Комплементарность
Conflict	Конфликт с другими потребностями
ResourcesRequired	Необходимые ресурсы
Products/ProcessesRequired	Необходимые продукты/процессы
ObservedDemand	Наблюдаемый спрос/поведение
SubjectiveAssessment	Субъективная оценка
ObjectiveIndicators	Объективные индикаторы
Confidence	Достоверность оценки
Weight w(t)	Нормированный вес в D(t), когда рассчитан

Методы

Метод	Назначение
detect()	Создать Need при наличии определяемого значимого разрыва состояния.
measureDeficit()	Рассчитать D по совместимым Actual/Desired/Reference State.
estimateUrgency()	Оценить скорость/тяжесть последствий неудовлетворения.

estimateSignificance()	Оценить значимость по принятой модели.
estimateSatisfiability()	Оценить возможность удовлетворения при известных ресурсах/технологиях.
matchResources()	Найти Resource/Product/Process, способные уменьшить дефицит.
measureSatisfaction()	Оценить изменение Need после интервенции/процесса.
updateWeight()	Передать параметры в алгоритм расчёта q_i и w_i .
resolveOverlap()	Выявить дублирующие/вложенные Need.
detectConflict()	Определить конфликт между потребностями и его тип.

Ограничения

Ограничение	Правило
Holder required	Need должна иметь Holder или явно определённый коллектив/экосистемный носитель.
State comparability	ActualState и Desired/ReferenceState должны быть сопоставимы или иметь явную функцию преобразования.
No negative deficit by default	$D \geq 0$; превышение желаемого уровня описывается Saturation/Surplus, если модель не задаёт иное.
Evidence separation	SubjectiveAssessment и ObjectiveIndicators хранятся отдельно и могут иметь разные веса.
No direct weight assignment	$w(t)$ не вводится вручную без provenance; он является результатом согласованного алгоритма.
Normalization external	$\sum_{i=1} w_i$ относится к выбранному срезу матрицы $D(t)$, а не к отдельной Need.
Overlap control	Пересекающиеся Need должны иметь отношения parent/child/overlap, чтобы избежать двойного счёта.
Temporal update	Изменение ActualState должно инициировать пересчёт зависимых параметров Need согласно правилам обновления.
Confidence propagation	Неопределённость исходных State/Indicators должна учитываться в Confidence параметров Need.

Ключевые связи

- heldBy -> Entity
- targets -> Entity/State
- definedBy -> State
- satisfiedBy -> Resource/Product/Process/Infrastructure
- conflictsWith -> Need
- complements -> Need
- partOf -> Need
- observedIn -> Location

4. Вычисляемый класс Subject

Subject определяется поверх общего множества Entity. Минимальный профиль субъектности $\Sigma = (B, P, D, A, F)$:

- В: устойчивость границы и идентичности
- Р: получение информации о среде/собственном состоянии
- D: выбор между альтернативными реакциями
- А: способность изменять состояние/среду
- F: обратная связь

Для более сложных субъектов используется профиль S с признаками E,A,M,L,G,C,R. Для каждого признака отдельно хранятся степень выраженности *v* и достоверность *c*; под *v* лежит более глубокий набор наблюдаемых параметров.

$$S_i = [[v_E, \dots, v_R], [c_E, \dots, c_R]]$$

5. Основные межклассовые связи

Источник	Связь	Цель
Entity	hasState	State
Entity	locatedIn	Location
Entity	participatesIn	Process
Entity	affectedBy	Event
Process	uses	Resource
Process	produces	Product
Process	changes	State
Infrastructure	supports	Process
Information	informs/controls	Process
LegalConstruct	regulates	Entity / Process / Resource
Need	heldBy	Entity / Subject
Need	definedByGapBetween	ActualState ↔ DesiredState
Need	satisfiedBy	Resource / Product / Process / Infrastructure
Event	changes	State
Location	contains	Entity

6. Эволюция онтологии

Онтология должна изменяться вместе с реальностью и научным знанием, но изменения не должны уничтожать историю модели. Для этого вводится versioned ontology evolution.

6.1. Типы изменений

Тип	Содержание
minor	Источник данных, единица, исправление ошибки, уточнение без изменения смысла класса.
structural	Добавление/удаление свойства, связи, метода, ограничения или класса.
fundamental	Изменение базового определения, принципа классификации или расчёта.

6.2. Структура записи изменения

$\Delta O = (type, target, old, new, reason, evidence, author/system, time, confidence, version, migration)$

- id
- type
- target
- old_definition
- new_definition
- reason
- evidence
- author_or_system
- timestamp
- confidence
- version_from
- version_to
- migration_rule
- status

6.3. Правила изменения

- Старая версия не уничтожается.
- Любое изменение имеет provenance.
- Deprecated элемент проходит active -> deprecated -> archived.
- По возможности задаётся migration rule для старых данных.
- AI/алгоритм может сформировать ChangeProposal; принятие фундаментальных изменений требует научной валидации.
- Изменения должны быть обратимо трассируемы до старой версии.

Рекомендуемый цикл:

$Reality \rightarrow Data \rightarrow Ontology \rightarrow Results \rightarrow Error/Novelty \rightarrow ChangeProposal \rightarrow Validation \rightarrow O^{(n+1)}$

7. Переход к матрице потребностей

Need является производным классом, связывающим Holder, ActualState и Desired/ReferenceState. После вычисления параметров Need формируется ненормированная оценка $q_i(t)$, а затем вес $w_i(t)$.

$$Need_i = f(Holder, X_{actual}, X_{desired}, context, constraints)$$

$$w_i(t) = q_i(t) / \sum_j q_j(t), \quad \sum_i w_i(t) = 1$$

Конкретная функция $q_i(t)$ остаётся отдельным следующим этапом научной модели.

8. Машиночитаемая спецификация

Параллельно с данным человекочитаемым документом онтология сохранена в YAML. В YAML представлены классы, наследование, свойства, методы, ограничения, связи, Subject и правила эволюции. Формат выбран как удобный одновременно для человека, программ и AI-агентов; при необходимости он может быть автоматически преобразован в JSON/JSON Schema или позже в RDF/OWL.

9. Дополнение и актуализация версии 3.1

Разделы 1–8 выше воспроизводят рабочую версию 2.0 без сокращений и используются как исторически трассируемая база. Настоящие разделы фиксируют согласованные после версии 2.0 расширения. При расхождении рабочих обозначений формулировки версии 3.1 имеют приоритет.

9.1. Принцип преемственности

- Базовый класс Entity, десять специализированных классов, их свойства, методы, ограничения, ключевые связи, вычисляемый Subject и правила эволюции онтологии сохраняются без отмены.
- Полная сеть отношений расширена до 23 формализованных блоков; отношения имеют тип, кратность, параметры, временную привязку, confidence и provenance.
- Рабочее обозначение $q_i(t)$ из раздела 7 версии 2.0 заменяется обозначением $N_i^{abs}(t)$, чтобы избежать неоднозначности и отделить абсолютную интенсивность потребности от нормированного веса.
- Низкая достоверность данных не трактуется как малая потребность: значение и confidence хранятся и вычисляются отдельно.

9.2. Полная система основных отношений

Универсальная запись отношения имеет источник, тип отношения, цель, параметры силы/эффекта, confidence, временную область и provenance. Для каждого блока ниже приводятся согласованные типы отношений и основные параметры. Кратность $n:m$ означает, что множество объектов каждого класса может быть связано с множеством объектов другого класса.

R = (source, relation, target, parameters, confidence, time, provenance)

ID блока	Классы	Кратность	Типы отношений	Параметры
entity_need	Entity ↔ Need	n:m	hasNeed, representsNeed, affectedByNeed, generatesNeed	membership_strength, confidence, intensity, dependency, time, scope, provenance
need_state	Need ↔ State	n:m	derivedFromState, targetsState, changesState, resolvedByStateTransition	deficit, urgency, reversibility, threshold, confidence
need_resource	Need ↔ Resource	n:m	satisfies, partiallySatisfies, enablesSatisfaction, substitutesFor	effectiveness, required_quantity, availability, substitutability, confidence, time, minimum_threshold, saturation_threshold
resource_process	Resource ↔ Process	n:m	input, catalyst, energy, tool, infrastructure, information, labor, reserve	quantity, criticality, substitutability, reusability, loss_rate, efficiency, confidence, time
process_product	Process ↔ Product	n:m	produces, transformsInto, assembles, modifies, repairs, destroys, recycles	quantity, quality, durability, usability, defect_rate, confidence, time
product_need	Product ↔ Need	n:m	directlySatisfies, enablesSatisfaction, partiallySatisfies, negativelyAffects	effect, coverage, speed, reach, duration, confidence, time
product_state	Product ↔ State	n:m	changesState, maintainsState, restoresState, preventsDeterioration, degradesState, hasNoMeasuredEffect	delta_state, effect, latency, duration, confidence, context
state_event	State ↔ Event	n:m	triggersStateChange, causesTransition, terminatesState, initiatesState,	delta_state, causal_strength, causal_confidence, confidence, time, context

ID блока	Классы	Кратность	Типы отношений	Параметры
			modifiesState, observesStateChange	
event_process	Event ↔ Process	n:m	initiates, terminates, pauses, resumes, accelerates, slows, modifies, disrupts, resultsFrom	effect, trigger_strength, causal_strength, causal_confidence, latency, duration_of_effect, confidence
process_state	Process ↔ State	n:m	improvesState, degradesState, maintainsState, stabilizesState, destabilizesState, transformsState, restoresState	direction, rate, duration, delta_state, confidence, context
entity_resource	Entity ↔ Resource	n:m	owns, controls, uses, consumes, hasAccessTo, stores, produces, transfers, requires, isExcludedFrom	quantity, ownership, control, access, usability, priority, claim_strength, allocation_share, confidence, time
entity_product	Entity ↔ Product	n:m	produces, owns, controls, uses, consumes, hasAccessTo, transfers, receives, stores, disposesOf, benefitsFrom, isHarmedBy	quantity, ownership_share, access_level, usage_intensity, benefit_effect, confidence, time
entity_infrastructure	Entity ↔ Infrastructure	n:m	owns, controls, operates, uses, hasAccessTo, maintains, finances, dependsOn, providesAccessTo, isExcludedFrom	access, control, utilization, confidence, time
entity_location	Entity ↔ Location	n:m	locatedAt, occupies, operatesIn, originatesFrom, movesThrough, hasAccessTo, isRestrictedFrom, ownsLocation, controlsLocation, dependsOnLocation	distance, travel_time, accessibility, confidence, time
entity_information	Entity ↔ Information	n:m	creates, owns, stores, accesses, uses, transmits, receives, modifies, verifies, dependsOn, isRestrictedFrom, isRepresentedBy	access, interpretability, relevance, usage, confidence, time
entity_legal_construct	Entity ↔ LegalConstruct	n:m	holdsRight, hasObligation, ownsByLaw, isPartyTo, isBeneficiaryOf, isLiableUnder, isRestrictedBy, isAuthorizedBy, isProtectedBy, hasClaimAgainst	strength, enforceability, confidence, valid_from, valid_to
entity_state	Entity ↔ State	1:n over time	hasState, entersState, leavesState, maintainsState, isCharacterizedBy, hasReferenceState	confidence, time, stability, rate_of_change
entity_event	Entity ↔ Event	n:m	participatesIn, causes, triggers, isAffectedBy, observes, prevents, isCreatedBy, isTerminatedBy, isTransferredBy, isModifiedBy	role, effect, causal_strength, causal_confidence, relevance, confidence, time
entity_process	Entity ↔ Process	n:m	executes, participatesIn, controls, initiates, terminates, benefitsFrom, isHarmedBy, providesInputTo, receivesOutputFrom, dependsOn	participation_intensity, effect, confidence, time
infrastructure_process	Infrastructure ↔ Process	n:m	enables, supports, hosts,	capacity, availability,

ИД блока	Классы	Кратность	Типы отношений	Параметры
			constrains, limitsCapacityOf, isRequiredBy, isUsedBy, isDegradedBy	utilization, criticality, confidence, time
location_resource_process	Location ↔ Resource, Process	n:m	resource: contains, stores, providesAccessTo, isSourceOf, isDestinationOf, restrictsAccessTo; process: hosts, permits, restricts, enables, isRequiredFor, isAffectedBy	quantity, access, suitability, capacity, confidence, time
information_process	Information ↔ Process	n:m	isInputTo, guides, controls, triggers, constrains, isProducedBy, isUpdatedBy, isValidatedBy, isRequiredBy	reliability, relevance, freshness, interpretability, confidence, time
legal_construct_resource_product_process	LegalConstruct ↔ Resource, Product, Process	n:m	resource: definesOwnershipOf, grantsAccessTo, restrictsUseOf, createsClaimOn, permitsTransferOf, encumbers; product: definesOwnershipOf, licenses, restrictsDistributionOf, grantsUsageRight, createsLiabilityFor; process: authorizes, requires, prohibits, regulates, obligates, limits, createsLiabilityFor	strength, enforceability, confidence, valid_from, valid_to

9.3. Параметры и правила отдельных отношений

9.3.1. Entity ↔ Need

Кратность: n:m

Типы: hasNeed, representsNeed, affectedByNeed, generatesNeed

Параметр	Смысл/диапазон
membership_strength	[0,1]
confidence	[0,1]
intensity	[0,1]
dependency	[0,1]
time	timestamp/interval
scope	['individual', 'group', 'collective']
provenance	required

- **Ограничение:** Need must reference at least one holder/represented entity where applicable
- **Ограничение:** Need is not identical to preference

9.3.2. Need ↔ State

Кратность: n:m

Типы: derivedFromState, targetsState, changesState, resolvedByStateTransition

Параметр	Смысл/диапазон
deficit	[0,1] after normalization
urgency	[0,1]
reversibility	[0,1]
threshold	optional numeric/interval
confidence	[0,1]

$$\text{Need}_i = f(\text{X_actual}, \text{X_desired})$$

$$\text{D}_i = d(\text{X_actual}, \text{X_desired})$$

- Distinguish NeedType from NeedInstance(t)
- Desired state may contain biological, social/normative and subjective components

9.3.3. Need ↔ Resource

Кратность: n:m

Типы: satisfies, partiallySatisfies, enablesSatisfaction, substitutesFor

Параметр	Смысл/диапазон
effectiveness	[0,1]
required_quantity	numeric
availability	[0,1]
substitutability	[0,1]
confidence	[0,1]
time	timestamp/interval
minimum_threshold	optional numeric
saturation_threshold	optional numeric

$$\text{S}_N = f(\text{q}_R)$$

$$\text{marginal_satisfaction} = d\text{S}_N/d\text{q}_R$$

9.3.4. Resource ↔ Process

Кратность: n:m

Типы: input, catalyst, energy, tool, infrastructure, information, labor, reserve

Параметр	Смысл/диапазон
quantity	numeric
criticality	[0,1]
substitutability	[0,1]
reusability	[0,1]
loss_rate	[0,1]
efficiency	context-dependent
confidence	[0,1]
time	timestamp/interval

- A Resource may also be an output of a Process

9.3.5. Process ↔ Product

Кратность: n:m

Типы: produces, transformsInto, assembles, modifies, repairs, destroys, recycles

Параметр	Смысл/диапазон
quantity	numeric
quality	[0,1] or domain scale
durability	domain-specific
usability	[0,1]
defect_rate	[0,1]
confidence	[0,1]
time	timestamp/interval

- Product may later also be classified as Resource

9.3.6. Product ↔ Need

Кратность: n:m

Типы: directlySatisfies, enablesSatisfaction, partiallySatisfies, negativelyAffects

Параметр	Смысл/диапазон
effect	[-1,1]
coverage	[0,1]
speed	time to effect
reach	number/share of affected entities
duration	effect duration
confidence	[0,1]
time	timestamp/interval

$$\Delta D_N = -f(\text{Product}, \text{Need})$$

9.3.7. Product ↔ State

Кратность: n:m

Типы: changesState, maintainsState, restoresState, preventsDeterioration, degradesState, hasNoMeasuredEffect

Параметр	Смысл/диапазон
delta_state	domain-specific vector
effect	[-1,1]
latency	time
duration	time
confidence	[0,1]
context	structured context

$$\Delta X = X(t1) - X(t0)$$

$$\Delta D_N = d(X(t1), X_desired) - d(X(t0), X_desired)$$

9.3.8. State ↔ Event

Кратность: n:m

Типы: triggersStateChange, causesTransition, terminatesState, initiatesState, modifiesState, observesStateChange

Параметр	Смысл/диапазон
delta_state	domain-specific
causal_strength	[0,1]
causal_confidence	[0,1]
confidence	[0,1]
time	timestamp
context	structured context

9.3.9. Event ↔ Process

Кратность: n:m

Типы: initiates, terminates, pauses, resumes, accelerates, slows, modifies, disrupts, resultsFrom

Параметр	Смысл/диапазон
effect	[-1,1]
trigger_strength	[0,1]
causal_strength	[0,1]
causal_confidence	[0,1]
latency	time
duration_of_effect	time
confidence	[0,1]

9.3.10. Process ↔ State

Кратность: n:m

Типы: improvesState, degradesState, maintainsState, stabilizesState, destabilizesState, transformsState, restoresState

Параметр	Смысл/диапазон
direction	[-1,1]
rate	dX/dt
duration	time
delta_state	domain-specific vector
confidence	[0,1]
context	structured context

$$dX/dt = F(\text{Process}, X, \text{Context})$$

$$\Delta X = \text{integral}(F(P, X, t), t_0..t_1)$$

9.3.11. Entity ↔ Resource

Кратность: n:m

Типы: owns, controls, uses, consumes, hasAccessTo, stores, produces, transfers, requires, isExcludedFrom

Параметр	Смысл/диапазон
quantity	numeric
ownership	[0,1]
control	[0,1]
access	[0,1]
usability	[0,1]
priority	domain-specific
claim_strength	[0,1]
allocation_share	[0,1]
confidence	[0,1]
time	timestamp/interval

$$\text{EffectiveResource} = \text{Quantity} * \text{Access} * \text{Usability}$$

9.3.12. Entity ↔ Product

Кратность: n:m

Типы: produces, owns, controls, uses, consumes, hasAccessTo, transfers, receives, stores, disposesOf, benefitsFrom, isHarmedBy

Параметр	Смысл/диапазон
quantity	numeric
ownership_share	[0,1]
access_level	[0,1]
usage_intensity	[0,1]
benefit_effect	[-1,1]
confidence	[0,1]
time	timestamp/interval

9.3.13. Entity ↔ Infrastructure

Кратность: n:m

Типы: owns, controls, operates, uses, hasAccessTo, maintains, finances, dependsOn, providesAccessTo, isExcludedFrom

Параметр	Смысл/диапазон
access	[0,1]
control	[0,1]
utilization	[0,1]
confidence	[0,1]
time	timestamp/interval

$$\text{EffectiveCapacity} = \text{Capacity} * \text{Availability} * \text{Accessibility} * \text{Reliability}$$

9.3.14. Entity ↔ Location

Кратность: n:m

Типы: locatedAt, occupies, operatesIn, originatesFrom, movesThrough, hasAccessTo, isRestrictedFrom, ownsLocation, controlsLocation, dependsOnLocation

Параметр	Смысл/диапазон
distance	numeric
travel_time	time
accessibility	[0,1]
confidence	[0,1]
time	timestamp/interval

9.3.15. Entity ↔ Information

Кратность: n:m

Типы: creates, owns, stores, accesses, uses, transmits, receives, modifies, verifies, dependsOn, isRestrictedFrom, isRepresentedBy

Параметр	Смысл/диапазон
access	[0,1]
interpretability	[0,1]
relevance	[0,1]
usage	[0,1]
confidence	[0,1]
time	timestamp/interval

$$\text{EffectiveInformation} = \text{Access} * \text{Interpretability} * \text{Relevance} * \text{Reliability}$$

9.3.16. Entity ↔ LegalConstruct

Кратность: n:m

Типы: holdsRight, hasObligation, ownsByLaw, isPartyTo, isBeneficiaryOf, isLiableUnder, isRestrictedBy, isAuthorizedBy, isProtectedBy, hasClaimAgainst

Параметр	Смысл/диапазон
strength	[0,1]
enforceability	[0,1]
confidence	[0,1]
valid_from	timestamp
valid_to	timestamp/nullable

9.3.17. Entity ↔ State

Кратность: 1:n over time

Типы: hasState, entersState, leavesState, maintainsState, isCharacterizedBy, hasReferenceState

Параметр	Смысл/диапазон
confidence	[0,1]
time	timestamp/interval
stability	[0,1]

Параметр	Смысл/диапазон
rate_of_change	dX/dt

9.3.18. Entity ↔ Event

Кратность: n:m

Типы: participatesIn, causes, triggers, isAffectedBy, observes, prevents, isCreatedBy, isTerminatedBy, isTransferredBy, isModifiedBy

Параметр	Смысл/диапазон
role	categorical
effect	[-1,1]
causal_strength	[0,1]
causal_confidence	[0,1]
relevance	[0,1]
confidence	[0,1]
time	timestamp

9.3.19. Entity ↔ Process

Кратность: n:m

Типы: executes, participatesIn, controls, initiates, terminates, benefitsFrom, isHarmedBy, providesInputTo, receivesOutputFrom, dependsOn

Параметр	Смысл/диапазон
participation_intensity	[0,1]
effect	[-1,1]
confidence	[0,1]
time	timestamp/interval

9.3.20. Infrastructure ↔ Process

Кратность: n:m

Типы: enables, supports, hosts, constrains, limitsCapacityOf, isRequiredBy, isUsedBy, isDegradedBy

Параметр	Смысл/диапазон
capacity	numeric
availability	[0,1]
utilization	[0,1]
criticality	[0,1]
confidence	[0,1]
time	timestamp/interval

9.3.21. Location ↔ Resource, Process

Кратность: n:m

resource: contains, stores, providesAccessTo, isSourceOf, isDestinationOf, restrictsAccessTo

process: hosts, permits, restricts, enables, isRequiredFor, isAffectedBy

Параметр	Смысл/диапазон
quantity	numeric when Resource
access	[0,1]
suitability	[0,1] when Process
capacity	numeric/domain-specific
confidence	[0,1]
time	timestamp/interval

9.3.22. Information ↔ Process

Кратность: n:m

Типы: isInputTo, guides, controls, triggers, constrains, isProducedBy, isUpdatedBy, isValidatedBy, isRequiredBy

Параметр	Смысл/диапазон
reliability	[0,1]
relevance	[0,1]
freshness	[0,1]
interpretability	[0,1]
confidence	[0,1]
time	timestamp/interval

$$\text{InformationEffectiveness} = \text{Reliability} * \text{Relevance} * \text{Freshness} * \text{Interpretability}$$

9.3.23. LegalConstruct ↔ Resource, Product, Process

Кратность: n:m

resource: definesOwnershipOf, grantsAccessTo, restrictsUseOf, createsClaimOn, permitsTransferOf, encumbers

product: definesOwnershipOf, licenses, restrictsDistributionOf, grantsUsageRight, createsLiabilityFor

process: authorizes, requires, prohibits, regulates, obligates, limits, createsLiabilityFor

Параметр	Смысл/диапазон
strength	[0,1]
enforceability	[0,1]
confidence	[0,1]
valid_from	timestamp
valid_to	timestamp/nullable

10. Иерархия потребностей и построение матрицы D_R(t)

Need остаётся производным классом, связывающим Holder, фактическое состояние и желаемое/референтное состояние. Версия 3.1 уточняет его иерархию и разделяет абсолютную интенсивность потребности, общий уровень неудовлетворённости и относительную структуру матрицы.

10.1. Иерархия Need

NeedType → Subcategory → NeedInstance(Entity, t)

NeedType задаёт устойчивый тип потребности; Subcategory детализирует его; NeedInstance является конкретным временным экземпляром потребности отдельного Entity, группы или другого допустимого носителя.

Рабочий верхний уровень NeedType версии 3.1:

1. **Поддержание существования** (ExistenceMaintenance)
2. **Физическая и функциональная целостность** (PhysicalFunctionalIntegrity)
3. **Безопасность** (Safety)
4. **Среда и пространство** (EnvironmentAndSpace)
5. **Доступ к ресурсам** (AccessToResources)
6. **Информация и познание** (InformationAndCognition)
7. **Автономия и действие** (AutonomyAndAction)
8. **Социальные потребности** (SocialNeeds)

9. **Воспроизводство и продолжение** (ReproductionAndContinuation)
10. **Развитие и увеличение возможностей** (DevelopmentAndCapabilityExpansion)
11. **Положительный субъективный опыт** (PositiveSubjectiveExperience)
12. **Самореализация, творчество и смысл** (SelfRealizationCreativityMeaning)

Этот перечень является рабочим верхним уровнем, а не закрытым окончательным каталогом. Изменения проходят общий механизм versioned ontology evolution.

10.2. Абсолютная интенсивность потребности

$$N_i^{\text{abs}}(t) = f(D_i, H_i, U_i, P_i, Dep_i, S_i, R_i)$$

Компонент	Содержание
D _i	Дефицит: расстояние между фактическим и желаемым/референтным состоянием.
H _i	Тяжесть последствий неудовлетворения.
U _i	Срочность / время до существенного ущерба.
P _i	Масштаб: число или доля затронутых Entity.
Dep _i	Зависимость нормального существования или функционирования от удовлетворения потребности.
S _i	Субъективная значимость и наблюдаемые/выявленные предпочтения.
R _i	Необратимость последствий.

Функциональная форма f пока не фиксируется окончательно: остаётся открытым выбор аддитивного, мультипликативного, fuzzy-rule или гибридного взаимодействия компонентов.

10.3. Confidence как отдельный слой

$$C_i(t) \in [0,1]$$

$C_i(t)$ характеризует достоверность оценки $N_i^{\text{abs}}(t)$. Он не входит автоматически как понижающий множитель в интенсивность потребности: отсутствие или низкое качество данных не означает отсутствия потребности.

10.4. Агрегация и нормирование

$$N^{\text{abs}} = N^{\text{abs}}(i, g, l, t)$$

Полная структура допускает измерения по NeedType i , группе Entity g , Location l и времени t . NeedInstance агрегируются по выбранному срезу до нормирования.

$$H_R(t) = \sum_i N_i^{\text{abs}}(t)$$

$H_R(t)$ — общий абсолютный уровень неудовлетворённых потребностей выбранной референтной системы R .

$$w_{i,R}(t) = N_i^{\text{abs}}(t) / \sum_j N_j^{\text{abs}}(t), \text{ если } H_R(t) > 0$$

$$D_R(t) = (w_{1,R}(t), \dots, w_{n,R}(t)), \sum_i w_{i,R}(t) = 1$$

Если $H_R(t)=0$, все представленные в выбранном срезе потребности удовлетворены. В этом случае нормирование $D_R(t)$ имеет нулевой знаменатель и относительный профиль считается неопределённым/неинформативным до появления положительной абсолютной интенсивности.

11. Живая онтология

Живая онтология — это операционный слой, в котором формальная схема наполняется конкретными экземплярами Entity, наблюдениями, состояниями, отношениями, fuzzy-принадлежностями и provenance и обновляется при поступлении новых данных.

11.1. Структура экземпляра

Поле	Назначение
entity_id	globally unique identifier
classes	структурированный map/record
properties	структурированный map/record
relations	список типизированных записей
states	список типизированных записей

Каждое динамическое или вычисленное значение сохраняется вместе с временем, confidence и provenance; старые наблюдения не перезаписываются без истории изменений.

11.2. Поток обновления

13. ingest_data
14. validate_schema
15. map_to_entities
16. update_properties
17. update_relations
18. update_states
19. instantiate_or_update_need_instances
20. compute_need_parameters
21. compute_N_i_abs
22. aggregate_need_instances
23. compute_H_R
24. normalize_to_D_R_if_H_R_positive
25. recalculate_fuzzy_memberships
26. recalculate_confidence
27. detect_constraint_violations
28. record_provenance
29. version_snapshot

Data → Ontology → NeedInstance → N^{abs} → Aggregation → H_R → D_R(t)

11.3. Принцип минимально достаточной модели

Живая онтология не требует полного цифрового описания каждого объекта мира. Базовый режим может использовать модели уровня класса и обновлять индивидуальные параметры только там, где это влияет на текущие состояния, отношения, NeedInstance или расчёт D_R(t). Это соответствует принципам class-level default, индивидуализации при необходимости и вычисления только затронутых зависимостей.

12. Машиночитаемая спецификация версии 3.1

Человекочитаемый документ синхронизирован с master-спецификацией версии 3.1. Машиночитаемая форма хранится параллельно в YAML и JSON. Она включает базовую архитектуру, классы, свойства, методы, ограничения, отношения, правила эволюции, модель потребностей и pipeline живой онтологии.

- ontology_master_v3_1.yaml

- ontology_master_v3_1.json
- ontology_relations_v1.yaml
- ontology_relations_v1.json

12.1. Изменения от версии 2.0 к 3.1

- Added NeedType → Subcategory → NeedInstance hierarchy
- Added 12 working top-level NeedType classes
- Replaced temporary Q_i need notation with $N_i_{abs}(t)$
- Added $H_R(t)$ as total absolute unmet-need level
- Separated $C_i(t)$ confidence from need intensity
- Added working component set $D_i, H_i, U_i, P_i, Dep_i, S_i, R_i$
- Added aggregation and normalization rules for $D_R(t)$
- Defined behavior for $H_R(t)=0$
- Updated live ontology data pipeline

12.2. Нормативный статус версий

Текст версии 2.0 сохранён в настоящем документе полностью для трассируемости. Для новых вычислений матрицы потребностей приоритет имеют уточнения разделов 9–12 версии 3.1. Удаление старой формулировки не требуется: механизм provenance и versioning позволяет восстановить, какая версия определения использовалась при каждом расчёте.